KISS Sorcar: A Relentless, Self-Continuing AI Agent for Integrated Software Development

Koushik Sen
EECS Department, UC Berkeley
ksen@berkeley.edu

Abstract

We present KISS Sorcar, an AI-powered general assistant and integrated development environment built around a layered agent architecture. At its foundation, the KISS Agent implements a budget-aware ReAct loop with native function calling. The Relentless Agent extends this with automatic multi-session continuation, enabling arbitrarily long tasks that survive context-window exhaustion. The Sorcar Agent adds coding tools, browser automation, and parallel sub-agent execution. The Chat Sorcar Agent introduces persistent multi-turn chat sessions, and the Worktree Sorcar Agent isolates every task on a dedicated git branch via worktrees so the developer's main working tree is never modified. We describe the design of each layer, the structured system prompt that governs agent behavior, and the engineering decisions that make the system practical for real-world software development.

1 Introduction

Large language models (LLMs) have demonstrated remarkable ability to generate code, reason about software architecture, and operate developer tools [Chen et al., 2021, Rozière et al., 2023]. A growing body of work has explored how to harness these capabilities for autonomous software engineering, from single-session agents that resolve GitHub issues [Yang et al., 2024, Wang et al., 2024] to industrial products marketed as AI teammates [GitHub, 2021, Cursor, 2024, Cognition Labs, 2024]. Yet using an LLM as a practical software engineering assistant still exposes several stubborn gaps: context windows are finite, a single mistake can derail an entire session, and generated changes are difficult to review or revert once applied to a live codebase.

We address these gaps through a five-layer agent hierarchy in which each layer adds exactly one concern:

1. **KISS Agent** — budget-tracked ReAct [Yao et al., 2023] loop with native function calling.
2. **Relentless Agent** — automatic continuation across sub-sessions.
3. **Sorcar Agent** — coding tools, browser automation, and parallel sub-agent execution.
4. **Chat Sorcar Agent** — persistent multi-turn chat sessions with history recall.
5. **Worktree Sorcar Agent** — git-worktree isolation so every task runs on its own branch.

The name "KISS" reflects the *Keep It Simple, Stupid* design philosophy: each layer is small, each concern is isolated, and the overall system avoids unnecessary abstraction. The name "Sorcar" pays homage to P. C. Sorcar, the legendary Indian magician, evoking the idea of an agent that performs feats that appear magical yet are grounded in disciplined engineering.

We realize this architecture as a free, open-source Visual Studio Code extension with full browser automation, multimodal support via a built-in Chromium instance, and real-time cost tracking. The user supplies their own API key, and the system handles the rest. Section 5 describes the extension's unique features in detail.

Preprint.

The system prompt as a first-class artifact. Recent empirical studies have shown that project-level instruction files—such as `.cursorrules`, `AGENTS.md`, or `CLAUDE.md`—significantly improve agent performance [Chatlatanagulchai et al., 2025, Mohsenimofidi et al., 2025]. We take this idea further: the system prompt is not a static preamble but a carefully engineered document that encodes invariants about code quality, verification discipline, and self-improvement. We drew initial inspiration from the GetShitDone prompt engineering collection [GetShitDone, 2025], which demonstrated that prescriptive prompts can make coding agents substantially more effective, and we iteratively refined the prompt through months of daily use on a production codebase (Section 4).

Quality over speed. We deliberately prioritize output quality over latency. In our experience, using a weaker or cheaper model often forces the developer to discard the agent's work and retry, ultimately increasing the total cost of completing a task. Conversely, giving a frontier model adequate time to validate its own output—running linters, type checkers, and tests before declaring success—dramatically reduces the "slop" (low-quality, subtly incorrect code) that plagues faster but less thorough agents. We expect token costs and inference latencies to continue falling [Gao et al., 2025], which will make this quality-first posture increasingly practical. In the meantime, the code produced by our agent is consistently well-organized, simple, and idiomatic.

Self-hosting as a stress test. We built the system using itself. The entire codebase—the KISS Agent framework, the Sorcar agent layers, the VS Code extension, and the system prompt—was developed and maintained by the agent operating on its own repository. This self-hosting discipline provides a continuous integration-style stress test: if the agent introduces a bug that impairs its own ability to work, the feedback loop is immediate. The simplicity of the layered architecture was both a prerequisite for and a consequence of this bootstrapping process. The four core agent classes are remarkably compact: the KISS Agent comprises 409 lines, the Sorcar Agent 323 lines, the Chat Sorcar Agent 120 lines, and the Worktree Sorcar Agent 692 lines—a total of roughly 1,500 lines of code (excluding empty lines and comments).

Evaluation. We evaluate on Terminal Bench 2.0 and achieve a 62.1% overall pass rate (75.3% pass@any over 4 trials) using Claude Opus 4.6, comparing favorably to Claude Code (58%) and Cursor (62%) on the same benchmark (Section 6).

Outline. Section 2 describes the motivating design principles. Section 3 presents the five-layer agent architecture. Section 4 details the system prompt. Section 5 covers the VS Code extension. Section 6 reports preliminary evaluation results. Section 7 discusses related work, and Section 8 concludes.

2 Motivation and Key Design Principles

Finite context, infinite tasks. Even the largest context windows are insufficient for complex refactoring or multi-file bug hunts. Rather than hoping the model finishes before hitting the limit, we design the system from the ground up to *continue* across multiple sub-sessions, each with its own context window, while preserving a compressed summary of prior progress.

Budget accountability. Every API call costs money. We track input tokens, output tokens, cache hits, and cache writes at every step, compute a running dollar cost, and enforce per-task and global budget ceilings. If the agent exceeds the budget, it raises a hard error rather than silently accumulating charges.

Safe-by-default code changes. Applying AI-generated changes directly to a developer's working tree is risky: a bad edit can break a build, and reverting requires manual git archaeology. We eliminate this risk entirely with the worktree layer, which runs every task on a disposable git branch in a separate worktree directory. The developer reviews changes and explicitly chooses to merge or discard.

No mocks, no fakes, real integration tests. Our test suite enforces a strict policy: tests must exercise actual behavior, not mocked approximations. This philosophy extends to the agent's instructions — when the agent writes tests for user code, we instruct it to avoid mocks and achieve full branch coverage.

Relentless execution. We design the system to be *relentless*: it does not give up when a sub-session fails. Instead, it summarizes what happened and continues with a fresh strategy informed by a summary of all prior attempts.

3 Agent Architecture

The five agent layers form a strict inheritance chain. Each layer delegates upward for the concerns it does not own.

3.1 KISS Agent

The KISS Agent is the innermost execution unit. It implements a standard ReAct loop [Yao et al., 2023] with the following characteristics:

Native function calling. We register tools as ordinary Python callables. The agent builds an OpenAI-compatible tool schema once at setup time and caches it, avoiding redundant schema construction on every LLM call. A special `finish` tool signals task completion and returns the result to the caller.

Step, token, and budget tracking. At every step the agent extracts input and output token counts from the API response, computes the dollar cost using a per-model pricing table, and updates both a local budget counter and a global (cross-agent) budget counter protected by a class-level lock. The agent checks three limits before each step: per-agent budget, global budget, and maximum step count.

Error resilience. The agent retries transient API errors (rate limits, server errors) up to a configurable threshold of consecutive failures. It detects non-retryable errors (authentication failures, permission denials) by matching error type names and message substrings, and raises them immediately.

Non-agentic mode. When tools are not needed, the agent can run a single generation without the ReAct loop, which is useful for summarization or question-answering sub-tasks.

The KISS Agent is stateless across runs: each call to its run method resets the conversation, token counters, and tool registry. This makes it safe to reuse a single agent instance for multiple sequential tasks.

3.2 Relentless Agent

The Relentless Agent wraps a KISS Agent in a continuation loop. Its core contribution is the ability to execute tasks that exceed a single context window by breaking them into sub-sessions.

Rather than investing in context-compaction techniques, we adopt a simple continuation protocol: when a sub-session exhausts its context window or step budget, the agent produces a *structured summary* of every action taken so far—chronologically ordered, with explanations and relevant code snippets—and a fresh sub-session resumes from that summary. This approach is related in spirit to Reflexion [Shinn et al., 2023], which feeds verbal self-critiques back into subsequent trials, but operates at the granularity of full software engineering tasks rather than isolated reasoning problems. We found empirically that a naïve instruction to "summarize the current context" produced poor continuations; requiring a *step-by-step chronological account with code snippets* dramatically improved coherence across sub-sessions. A potential limitation is that summaries may grow unwieldy for multi-day tasks; in practice, we have not encountered this problem even for tasks spanning several hours, but a thorough evaluation of summary scaling remains future work.

Continuation protocol. The `finish` tool exposed to the inner KISS Agent accepts three fields: a success flag, a continue flag, and a summary. When the agent sets `is_continue=True`, the Relentless Agent starts a new sub-session with a fresh context window. The prompt for the new session includes a chronologically ordered list of all prior attempt summaries, instructing the agent not to redo completed work. The continuation prompt template is:

```
# Task Progress (Continuation {continuation_number})

{progress_text}

# Continue
- Complete the rest of the task.
- **DON'T** redo completed work.
- If you have been retrying the same approach without progress,
  step back and rethink the strategy from scratch.
```

Forced continuation on failure. If a sub-session raises an exception (e.g., the step limit is hit before the agent calls finish), the Relentless Agent does not abort. Instead, it saves the full trajectory to a temporary file, spawns a separate summarizer agent to read the trajectory and produce a concise summary, and then uses that summary as the progress text for the next sub-session. This ensures that even crashed sessions contribute useful context to subsequent attempts. The summarizer receives the following prompt:

```
# Summarizer

The trajectory of the agent is stored in the file: {trajectory_file}

# Instructions
- Read the trajectory file and analyze it.  The trajectory file
  could be large.
- Return a precise chronologically-ordered list of things the
  agent did with the reason for doing that along with relevant
  code snippets
```

To force the agent to self-continue before hitting the step limit, we augment the system prompt with an instruction that fires near the end of the budget:

```
# MOST IMPORTANT INSTRUCTIONS
- **At step {step_threshold}: you MUST call
  finish(success=False, is_continue=True,
  summary="precise chronologically-ordered list of things
  the agent did with the reason for doing that along with
  relevant code snippets")** or if the task is not complete
  and you are at risk of running out of steps or context
  length.
- Work dir: {work_dir}
- Current process PID: {current_pid} -- NEVER kill this
  process.
```

Budget aggregation. We accumulate token counts and dollar costs across all sub-sessions. Each new sub-session receives offsets so that the usage display shown to the user reflects the true cumulative cost, not just the cost of the current sub-session.

3.3 Sorcar Agent

The Sorcar Agent adds the tools that make the system useful for software development and general-purpose automation.

Coding tools. We provide four core tools: a shell command executor with streaming output, a file reader, a precise string-based file editor, and a file writer. The shell executor supports a configurable timeout, streams output to the user interface in real time, and respects a stop event that allows the user to cancel a running command.

Browser automation. A web-use tool provides programmatic browser control: navigating to URLs, reading page accessibility trees, clicking elements, typing text, pressing keys, scrolling, and taking screenshots. This enables the agent to research documentation, verify deployed applications, and interact with web-based tools.

Parallel sub-agents. An optional parallel execution tool spawns independent Sorcar Agent instances in a thread pool. Each sub-agent gets its own LLM context and tool set. This is useful for embarrass-

ingly parallel tasks such as summarizing multiple files or researching independent topics. We collect the results and return them in input order.

User interaction. An ask-user-question tool allows the agent to pause execution and request clarification from the user. In the VS Code integration, this renders as a text input in the sidebar; in CLI mode, it reads from standard input.

Docker isolation. When a Docker image is specified, we replace the coding tools with Docker-aware variants that execute commands inside a container, providing an additional layer of sandboxing for untrusted tasks.

3.4 Chat Sorcar Agent

The Chat Sorcar Agent adds multi-turn conversation persistence.

Chat sessions. We assign each task to a chat session identified by a stable chat ID. The agent persists tasks and their results to a local database. When a new task arrives within the same chat session, the agent loads all prior tasks and results and prepends them to the prompt as numbered context entries, allowing the LLM to reference earlier work.

Session management. The agent supports three operations: starting a new chat (which clears the chat ID), resuming a chat by task description (which looks up the corresponding chat ID), and resuming by explicit chat ID. This enables both automatic session continuity in an IDE and manual session selection from the command line.

Metadata persistence. After each task, the agent records metadata including the model used, working directory, software version, token count, cost, and whether the task used parallel execution or worktree isolation. This audit trail supports cost analysis and debugging.

3.5 Worktree Sorcar Agent

The Worktree Sorcar Agent is the outermost layer and the one that users interact with in the VS Code extension. Its defining feature is git-worktree isolation.

Branch-per-task. When a task starts, the agent creates a new git branch and a corresponding worktree directory. The branch name encodes the chat ID and a timestamp for uniqueness. All agent modifications happen inside the worktree; the user's main working tree remains untouched.

Dirty-state preservation. If the user's main working tree has uncommitted changes, the agent copies them into the worktree and creates a baseline commit. This ensures the agent sees the same state the user sees, while keeping the user's actual index and working tree clean. During merge, we use cherry-pick from the baseline commit to replay only the agent's changes, excluding the dirty-state snapshot.

Auto-commit and auto-merge. When the user starts a new task or a new chat, we automatically commit any pending worktree from a previous task with an LLM-generated commit message and squash-merge it into the original branch. If the merge has conflicts, we preserve the branch and provide the user with instructions for manual resolution.

Concurrency safety. A per-repository file lock serializes the checkout, stash, merge, and pop sequence so that concurrent tabs in the IDE cannot interleave operations on the same repository. Thread-local storage isolates per-task state (stream parsing buffers, bash output buffers, recording state) so that stopping one task does not corrupt another.

Crash recovery. We store all worktree state in git itself (branch names, git config entries) rather than in sidecar files. On process restart, the agent queries git for any pending branch matching its chat ID prefix and reconstructs all instance attributes from git config, enabling seamless recovery.

Graceful fallback. If the working directory is not inside a git repository, if the repository has no commits, or if HEAD is detached, the agent falls back to direct execution without worktree isolation, ensuring it never fails due to git preconditions.

4 The System Prompt

The system prompt is a structured document that governs the agent's behavior across all tasks. It is not a generic instruction to "be helpful" but rather a precise specification of engineering discipline. We describe its key sections.

4.1 Execution Mindset

The prompt opens with a directive that sets the tone:

```
# FOCUS ON THE GIVEN TASK. ITS COMPLETION IS YOUR SOLE GOAL.
# BE RELENTLESS. BE CALM. BE RIGOROUS. BE ACCURATE.
# CHECK FACTS. NO AI SLOP.
```

Each directive addresses a specific failure mode observed in LLM-based agents:

- **"FOCUS ON THE GIVEN TASK. ITS COMPLETION IS YOUR SOLE GOAL."** LLMs have a tendency to drift: they comment on the difficulty of a problem, explore tangential concerns, or ask clarifying questions that the user has already answered. This directive anchors the model on the task at hand and discourages meta-commentary that consumes tokens without making progress.
- **"BE RELENTLESS."** When an agent encounters an error—a failing test, a lint violation, a command that returns unexpected output—the default LLM behavior is to apologize, summarize the failure, and ask the user what to do next. This directive instructs the model to treat errors as obstacles to overcome, not as reasons to stop.
- **"BE CALM."** The complement of relentlessness. An overly aggressive agent that panics on encountering an error may thrash between approaches without deliberation. This directive encourages the model to analyze errors methodically before reacting.
- **"BE RIGOROUS."** This instructs the model to follow the verification and testing disciplines encoded later in the prompt, rather than taking shortcuts when a solution *looks* right.
- **"BE ACCURATE."** LLMs frequently generate plausible-but-wrong code, file paths, or command-line flags. This directive raises the model's threshold for asserting facts, encouraging it to verify claims against the actual file system or documentation rather than relying on parametric memory.
- **"CHECK FACTS."** A more specific version of "be accurate": when the model is about to state a fact (e.g., "this function takes three arguments"), it should read the source rather than relying on its training data, which may be outdated or wrong.
- **"NO AI SLOP."** "AI slop" refers to the low-quality, generic, hedging text that LLMs produce when they lack confidence: filler phrases like "certainly!", unnecessary caveats, and boilerplate explanations that the user did not ask for. This directive discourages such output and encourages the model to be concise and substantive.

4.2 Identity and Tool Rules

We give the agent a concrete identity (name, developer, repository URL) rather than a vague persona. Tool usage rules are explicit and mechanical:

```
# Identity

- You are KISS Sorcar, an AI based General Assistant and
  Integrated Development Environment (IDE), developed by
  Koushik Sen (ksen@berkeley.edu)
- Your public repository is at
  https://github.com/ksenxx/kiss_ai

# Rules

- Write() for new files. Edit() for small changes.
- Run Bash commands synchronously using the
  `timeout_seconds` parameter. Use 300s (default) for
  commands. If a command times out, retry with a higher
  timeout. Only for commands expected to exceed 10 minutes,
  run in the background with output redirected to a file
  and poll periodically.
- Use go_to_url() for browser tool.
- Call finish(success=True, is_continue=False,
  summary="detailed summary of what was accomplished ...")
  once the task is complete and all Pre-Finish Verification
  checks have passed.
- **The user cannot see intermediate chat. Show whatever
  user asks in the summary of the finish tool call.**
- READ large files in chunks.
- Create temporary files in PWD/tmp
- Use ULTRA thinking always
- **If you are running out of context length or steps, DO
  NOT try to complete the task urgently, but continue the
  task by calling 'finish'**
- PWD in the system prompt and user prompt denotes current
  working directory.
```

The identity block gives the agent a concrete name, developer, and repository URL rather than a vague persona such as "you are a helpful assistant." This anchors the model's self-representation, reducing the likelihood of confused or contradictory self-references, and it provides a URL the model can share when asked about its source code.

Each tool rule addresses a specific failure mode:

- **"Write() for new files. Edit() for small changes."** Without this distinction, the model may use `Write()` to overwrite an existing file with a slightly modified version, losing content it forgot to include. By reserving `Write()` for new files and requiring `Edit()` for modifications, the instruction ensures that changes are surgical and that unchanged portions of a file are never at risk.
- **Bash timeout guidance.** LLMs frequently launch shell commands without considering their runtime. A compilation or test suite that takes five minutes will time out at the default 30-second shell timeout in most agent frameworks, causing spurious failures. The instruction to use 300 seconds as the default, retry with higher timeouts on timeout, and run long commands in the background with output redirected to a file provides a mechanical protocol that handles the common cases without requiring the model to estimate runtime from first principles.
- **"Use go_to_url() for browser tool."** The agent has access to multiple tools that could plausibly interact with the web (shell-based `curl`, the browser tool, etc.). This instruction eliminates ambiguity by specifying which tool to use for browser-based interactions.
- **"The user cannot see intermediate chat."** In the VS Code extension, the user sees only the final summary returned by the `finish` tool, not the intermediate chain-of-thought or tool calls. Without this instruction, the model may "tell" the user something in an intermediate message and then assume the user has seen it, leading to confusion when the user asks for information the model believes it already provided. The instruction forces the model to put all user-facing information into the `finish` summary.
- **"READ large files in chunks."** Reading a 10,000-line file in a single tool call consumes a large fraction of the context window. By instructing the model to read files in chunks, the prompt prevents context-window exhaustion on a single file read, preserving capacity for the rest of the task.
- **"Create temporary files in PWD/tmp."** Without this instruction, the model creates temporary files in unpredictable locations (the system `/tmp`, the home directory, or scattered

throughout the project). Centralizing temporary files in a known directory makes cleanup predictable and prevents polluting the project tree with artifacts.

- **"Use ULTRA thinking always."** This instruction activates the model's extended reasoning mode (also known as "thinking" or "chain-of-thought" mode), in which the model performs additional internal deliberation before producing a response. Extended reasoning is particularly valuable for complex multi-step tasks where the model must plan before acting.
- **"If you are running out of context length or steps, DO NOT try to complete the task urgently, but continue the task by calling finish."** When the context window is nearly full, LLMs exhibit a "rush to finish" behavior: they skip verification steps, make hasty edits, and call `finish` with an incomplete result. This instruction redirects that urgency into the continuation protocol (Section 3.2), ensuring that a clean handoff to a new sub-session produces better results than a frantic attempt to squeeze everything into the remaining tokens.
- **"PWD in the system prompt and user prompt denotes current working directory."** The acronym "PWD" appears throughout the prompt and in user task descriptions (e.g., "edit PWD/src/main.py"). Without this definition, the model might interpret PWD as the Unix environment variable `$PWD` and attempt to expand it, or misinterpret it as a literal directory name. The explicit definition ensures consistent interpretation.

A separate rule about the `PageDrop` API allows the agent to upload HTML files and share them via a public URL, enabling it to create and share rich reports, visualizations, or documentation.

4.3 Pre-flight Checks

Before modifying any file, we instruct the agent to read it first:

```
## Pre-flight Checks

- Read every file you will modify before changing it.
- If the task depends on existing architecture or behavior,
  read the relevant source files first.
- If the task references files, commands, or config that do
  not exist, stop and ask or report instead of guessing.
- **When fixing a bug, an issue, or a race, write tests to
  confirm them. Then fix them.**
```

Each pre-flight check targets a specific category of avoidable error:

- **"Read every file you will modify before changing it."** The most common source of agent-introduced bugs is modifying a file based on an incorrect assumption about its current contents. The model may "remember" an older version of the file from its training data, or it may extrapolate from a partial reading. By requiring a fresh read immediately before any edit, the instruction ensures that the model operates on the actual current state of the file, not a stale mental model.
- **"If the task depends on existing architecture or behavior, read the relevant source files first."** This extends the previous rule from individual files to architectural context. A task like "add a caching layer to the database module" requires understanding not just the file to be modified, but also how callers interact with it, what interfaces exist, and what invariants are assumed. The instruction prevents the model from jumping straight to code generation without understanding the broader context.
- **"If the task references files, commands, or config that do not exist, stop and ask or report instead of guessing."** LLMs have a strong tendency to confabulate: when asked to modify a file that does not exist, the model will often proceed as if it does, producing edits against phantom content. This instruction converts a silent failure (wrong edits applied to a nonexistent file, silently creating it) into an explicit clarification request.
- **"When fixing a bug, an issue, or a race, write tests to confirm them. Then fix them."** This instruction mandates a test-first discipline for bug fixes. The motivation is two-fold: first, a test that reproduces the bug provides concrete verification that the fix is correct (the test should pass after the fix and fail before it). Second, writing the test forces the model to understand the bug precisely before attempting a fix, reducing the risk of an ad hoc patch that addresses a symptom rather than the root cause.

4.4 Code Style Guidelines

The prompt encodes a minimalist code philosophy:

```
## Code Style Guidelines

- Write simple, clean, and readable code with minimal
  indirection
- Avoid unnecessary object attributes, local variables, and
  config variables
- Avoid tight coupling among files and modules.
- Avoid object/struct attribute redirections
- No redundant abstractions or duplicate code
- Public methods MUST have full documentation
- Understand the root cause of an issue or bug, and patch
  the root cause instead of an ad hoc superficial fix.
- Before you write code, wait and think if the code is
  simple, elegant, general, and minimal.
- Once you finish the task, DO NOT write documentations
  unless the task specifically requires it.
```

Each guideline addresses a specific anti-pattern commonly exhibited by LLM-generated code:

- **"Write simple, clean, and readable code with minimal indirection."** LLMs tend to over-engineer solutions, introducing unnecessary abstractions, helper classes, and levels of indirection. Simple code is easier to review, test, and maintain.
- **"Avoid unnecessary object attributes, local variables, and config variables."** LLMs frequently introduce intermediate variables that serve no purpose—for example, assigning a return value to a local variable only to immediately return it on the next line, or storing a constant in a configuration file when it is used in exactly one place. Each unnecessary variable adds a name the reader must track and a site where the value could be accidentally mutated.
- **"Avoid tight coupling among files and modules."** When the model adds a feature that touches multiple files, it may introduce imports, shared global state, or cross-module function calls that create tight coupling. This instruction encourages modular design where each module has a clear, narrow interface.
- **"Avoid object/struct attribute redirections."** An attribute redirection occurs when an object stores a reference to another object solely to forward method calls to it—for example, `self.x = other.x` at construction time, creating two paths to the same value. LLMs commonly introduce such redirections when adapting code from one context to another. They make it harder to reason about ownership and lifetime.
- **"No redundant abstractions or duplicate code."** LLMs sometimes create a new utility function or class that duplicates functionality already present in the codebase, because they did not read the existing code thoroughly. This instruction reminds the model to check for existing implementations before creating new ones, and to factor out duplication when it encounters it.
- **"Public methods MUST have full documentation."** While the prompt generally discourages unnecessary documentation (see the last item), public methods are the API surface that other developers and modules depend on. Documentation on public methods is not optional—it specifies the contract.
- **"Understand the root cause of an issue or bug, and patch the root cause instead of an ad hoc superficial fix."** LLMs frequently apply symptom-level fixes: adding a null check where the real problem is that a variable should never be null, or catching an exception where the real problem is that the caller passes invalid arguments. This instruction forces the model to trace the causal chain to the root and fix it there.
- **"Before you write code, wait and think if the code is simple, elegant, general, and minimal."** This is a metacognitive instruction that asks the model to pause and evaluate its plan before committing to an implementation. By explicitly requesting this pause, the prompt encourages the model to spend more inference-time compute on design, reducing the likelihood of producing a first-draft solution that works but is unnecessarily complex.
- **"Once you finish the task, DO NOT write documentations unless the task specifically requires it."** Without this instruction, the model frequently concludes a task by writing README updates, docstrings, and inline comments that the user did not ask for. This

busywork wastes tokens, creates maintenance burden, and may introduce inaccuracies in documentation that is not automatically validated.

4.5 Deep Work Rules

```
## Deep Work Rules

- When the task says "align", "match", or "make
  consistent", read the target to determine the exact
  target state before editing. Never edit based on a vague
  reference.
- Use concrete values, not indirections. Instead of "update
  X to match Y", first read Y, then write the specific
  values into X.
- For multi-part work, list the concrete planned changes
  before executing them.
- Every meaningful change should have a concrete
  verification method (test, grep, CLI command).
```

The deep work rules address a failure mode where the model interprets an instruction loosely and makes changes that are directionally correct but concretely wrong:

- **"When the task says 'align', 'match', or 'make consistent', read the target to determine the exact target state before editing."** When a user says "make file A consistent with file B," the model often reads file A, infers what file B probably contains, and edits A based on that inference—without ever reading B. This instruction mandates reading the target first, ensuring that the alignment is based on concrete facts rather than assumptions.
- **"Use concrete values, not indirections."** A related failure mode occurs when the model's plan says "update X to match Y" but the model never resolves what Y actually is. The instruction requires the model to first read Y, extract the specific values, and then write those values into X. This eliminates a class of errors where the model's mental model of Y differs from reality.
- **"For multi-part work, list the concrete planned changes before executing them."** When a task requires changes to multiple files, executing them one at a time without a plan leads to inconsistencies: the model may change a function signature in one file but forget to update a caller in another. Listing all planned changes before executing any of them forces the model to consider the full scope of the change and identify dependencies.
- **"Every meaningful change should have a concrete verification method."** A change without a verification method is a change that cannot be confirmed to work. This instruction requires the model to pair each change with a specific check—a test, a grep for the expected pattern, a CLI command that exercises the changed behavior—ensuring that the change can be validated programmatically rather than by visual inspection of a diff.

4.6 Planning for Complex Tasks

The planning instructions use a complexity threshold—three or more files, cross-module changes, or architectural work—to decide when formal planning is required:

```
## Planning for Complex Tasks

For tasks involving 3+ files, cross-module changes, or
architectural work:

1. List the files that need to change and why.
1. State the exact intended change in each file.
1. Identify dependencies and execution order.
1. State how each change will be verified.

For simple single-file tasks, skip formal planning and
execute directly.
```

Each planning step targets a specific failure mode:

- **"List the files that need to change and why."** This forces the model to enumerate the full blast radius of a change before touching any file. Without this step, the model often

discovers mid-task that additional files need changes, leading to incomplete or inconsistent modifications.

- **"State the exact intended change in each file."** Listing files alone is insufficient; the model must also articulate *what* will change in each file. This converts a vague plan ("update the database module") into a concrete specification ("add a `cache_ttl` parameter to `DatabaseClient.__init__`, modify the query method to check the cache before hitting the database, add a cache invalidation method").
- **"Identify dependencies and execution order."** Some changes must precede others: a new utility function must be written before callers can import it, a migration must run before code that depends on the new schema. Identifying these dependencies prevents the model from executing changes in an order that produces intermediate states where the code does not compile or tests do not pass.
- **"State how each change will be verified."** The verification requirement from the Deep Work Rules is reinforced here at the planning stage, ensuring that verification is planned alongside the changes rather than treated as an afterthought.

The escape clause—"for simple single-file tasks, skip formal planning and execute directly"—avoids the overhead of planning trivial changes. Requiring a formal plan for a one-line typo fix would waste tokens and slow down the agent without any compensating benefit.

4.7 Testing Instructions

The testing section is perhaps the most opinionated:

```
## Testing Instructions

- Run lint and typecheckers and fix any lint and typecheck
  errors.
- You MUST achieve 100% branch coverage
- Tests MUST NOT use mocks, patches, fakes, or any form of
  test doubles
- You MUST write integration tests
- Each test should be independent and verify actual behavior
- **Do NOT run all tests after modifications. Only run the
  impacted tests**
- To confirm a race condition, add sleep statements before
  racing statements with delays less than 0.1s
```

Each testing instruction addresses a specific concern:

- **"Run lint and typecheckers and fix any lint and typecheck errors."** Before committing any change, the agent must ensure it does not introduce lint violations or type errors. This catches a broad class of issues—unused imports, type mismatches, style violations—that would otherwise accumulate across tasks.
- **"You MUST achieve 100% branch coverage."** Full branch coverage ensures that every conditional path in the code under test has been exercised. LLMs tend to write happy-path tests that cover the main code path but ignore error handling, edge cases, and early-return branches. The 100% target forces the model to write tests for every branch, including error paths and boundary conditions.
- **"Tests MUST NOT use mocks, patches, fakes, or any form of test doubles."** This is the most opinionated rule. Mocks test that code calls certain methods in a certain order—they test the implementation, not the behavior. A test suite built on mocks can pass with flying colors while the system is fundamentally broken, because the mocks hide the real dependencies. Integration tests that exercise actual behavior are more expensive to run but provide genuine confidence that the system works.
- **"You MUST write integration tests."** This reinforces the no-mocks rule by explicitly requiring integration tests rather than unit tests. The distinction matters: a unit test in isolation may verify that a function produces the right output for a given input, but an integration test verifies that the function works correctly within the larger system—with real file I/O, real database connections, and real inter-module interactions.
- **"Each test should be independent and verify actual behavior."** Test independence means that running tests in any order produces the same results. Tests that depend on shared state or execution order are brittle and difficult to debug when they fail. "Verify actual behavior"

reiterates that tests should assert on observable outcomes (return values, side effects, system state) rather than implementation details.

- **"Do NOT run all tests after modifications. Only run the impacted tests."** Running the full test suite after every small change is wasteful when only a few modules are affected. For a large project, a full test run may take minutes, and doing it after every edit adds up to significant wasted time and compute. This instruction directs the model to identify which tests are affected by its changes and run only those, improving iteration speed.
- **"To confirm a race condition, add sleep statements before racing statements with delays less than 0.1s."** Race conditions are notoriously difficult to reproduce because they depend on precise timing. By inserting small sleep delays at strategic points, the model can widen the race window and make the bug manifest deterministically in a test. The 0.1-second upper bound keeps the test fast while still being sufficient to expose most races.

4.8 Self-Improvement Loop

The agent maintains a preferences file that captures user invariants discovered during task execution:

```
## Self-Improvement Loop

- Read the instructions in PWD/USER_PREFS.md at the start
  of each task.
- Then update PWD/USER_PREFS.md to capture the user
  preferences and invariants by analyzing the task. DO NOT
  ADD ANY CODE SNIPPETS OR SYMBOLS. You MUST carefully and
  thoroughly get rid of the user preferences and invariants
  that conflict with the newly added ones.
```

Each instruction in this section serves the goal of cross-session learning:

- **"Read the instructions in PWD/USER_PREFS.md at the start of each task."** The preferences file contains invariants learned from previous tasks—coding conventions, project-specific rules, architectural decisions. Reading it at the start of each task ensures that the agent's behavior is consistent across sessions, even though each session starts with a fresh context window.
- **"Then update PWD/USER_PREFS.md to capture the user preferences and invariants by analyzing the task."** After completing a task, the agent may have learned new information about the user's preferences: a preferred naming convention, a disliked pattern, a project-specific invariant. Writing these to the preferences file makes them available to future sessions. This mechanism allows the agent to accumulate project knowledge over time without requiring the user to repeat themselves.
- **"DO NOT ADD ANY CODE SNIPPETS OR SYMBOLS."** Code snippets in the preferences file are fragile: they become stale as the codebase evolves, and they consume tokens that would be better spent on natural-language descriptions of invariants. This instruction keeps the file compact and robust to code changes.
- **"You MUST carefully and thoroughly get rid of the user preferences and invariants that conflict with the newly added ones."** Over time, preferences may become contradictory—for example, an early preference might say "use camelCase for test methods" while a later correction says "use snake_case." Without explicit conflict resolution, the file would accumulate contradictions that confuse the agent. This instruction mandates that the agent actively resolve conflicts when updating the file, keeping it internally consistent.

4.9 Pre-Finish Verification

Before declaring a task complete, the agent must pass a structured verification checklist:

```
## Pre-Finish Verification

Before calling finish(success=True, ...), you MUST:

1. Re-read every file you modified and verify the changes
   are correct.
1. Run the required checks (lint, typecheck, tests) and
   fix any failures.
1. Explicitly check each user requirement against what was
   delivered.
1. If any check fails, continue working instead of
   finishing.
1. If you have retried the same fix 3 times without
   progress, step back, rethink the approach from scratch,
   and try a different strategy.
```

Each step in this checklist addresses a specific way agents declare premature success:

- **"Re-read every file you modified and verify the changes are correct."** This is the analog of a code review performed by the agent on its own work. The model may have introduced a typo, forgotten to close a bracket, or made an edit that looked correct in the diff but is wrong in the full file context. Re-reading the file after all edits are complete catches these errors.
- **"Run the required checks (lint, typecheck, tests) and fix any failures."** This converts the subjective assessment "I think my changes are correct" into an objective, automated verification. If the lint, typecheck, or test suite fails, the agent must fix the failure before declaring success.
- **"Explicitly check each user requirement against what was delivered."** The model may have completed a task that it *thinks* satisfies the user's request but actually misses a requirement. This instruction forces a systematic comparison between the original task description and the delivered result, catching gaps and misinterpretations.
- **"If any check fails, continue working instead of finishing."** Without this instruction, the model may call `finish(success=True)` even when it knows a check has failed, rationalizing that the failure is "minor" or "unrelated." The instruction makes the rule absolute: no finishing until all checks pass.
- **"If you have retried the same fix 3 times without progress, step back, rethink the approach from scratch, and try a different strategy."** LLMs can enter repetitive loops where they apply the same incorrect fix repeatedly, each time hoping for a different result. The three-retry threshold forces the model to break out of such loops by abandoning the current approach and reconsidering the problem from first principles. This is analogous to the debugging heuristic "if you've been staring at the same code for twenty minutes, you're looking in the wrong place."

4.10 Web Research Protocol

When the agent needs external knowledge, the prompt prescribes a structured research workflow rather than allowing ad-hoc browsing:

```
## Use web tools when you need to:

- When you need to collect knowledge from the internet,
  visit **AT LEAST 100 WEB SITES** and collect information
  necessary for the task without much thinking in a new
  file PWD/tmp/information-{unique_id}.md. Then go over
  information in PWD/tmp/information-{unique_id}.md, think
  deeply on how to complete the task at hand, and
  complete it.
- If you need to login to an website while browsing for
  information, you MUST ask the user to help you with
  login. You **MUST NOT** ask the user to login to Google.
```

The rationale is a two-phase separation between *collection* and *synthesis*. LLMs tend to anchor on the first few results they encounter, which biases their solutions toward a narrow slice of the design space. By forcing the agent to accumulate a broad set of information into a file *before* reasoning about it, the protocol counteracts anchoring bias and encourages the model to consider diverse approaches. The

"at least 100 web sites" threshold is deliberately aggressive: it ensures the agent does not shortcut the research phase after two or three hits, and the resulting information file serves as an auditable artifact of what the agent considered.

The login instruction addresses a practical obstacle in web research: many websites require authentication before revealing their content. Rather than silently skipping gated pages or hallucinating their contents, we instruct the agent to ask the user for help with login. The Google exception acknowledges that the agent can search the web without a Google account, and asking the user to log into Google would provide no benefit while creating unnecessary friction.

4.11 Desktop Application Control

The agent can interact with graphical desktop applications using screenshots, keyboard, and mouse:

```
## Launch desktop apps

- Use screenshots, keyboard, and mouse to control a
  desktop app.
- Do not launch VS Code or its extensions.
```

This instruction enables the agent to operate GUI applications (Preview, browsers, graphical diff tools) when command-line alternatives are insufficient. The explicit prohibition on launching VS Code prevents a recursive loop: since the agent runs *inside* a VS Code extension, launching another VS Code instance or modifying extension state from within the agent could corrupt the host session or create deadlocks.

4.12 Sorcar-Specific Overrides

A final section provides project-specific instructions that are injected when the agent operates on its own codebase:

```
## Sorcar-specific instructions:

- Use 'uv run check --full' to lint, typecheck, and format
  code.
- Run 'uv run pytest -v' with a timeout of 900 seconds to
  test KISS
- Do **NOT run all KISS tests after modifications. Only run
  the impacted tests**
- **Do NOT install the KISS Sorcar extension from inside
  Sorcar**
- If you mention a file in the chat, make the file
  clickable so that when the user clicks, it opens in the
  editor of VSCode
- Read PWD/SORCAR.md and treat its contents as
  instructions and allow those instructions to override the
  instructions above
```

These overrides serve three purposes. First, they specify the exact toolchain commands for the KISS project itself (`uv run check --full`, `uv run pytest`), eliminating guesswork about which linter, formatter, or test runner to use. Second, they impose efficiency constraints: running the full test suite after every small change is wasteful when only a few modules are affected, so we instruct the agent to scope test execution to impacted tests. Third, they prevent dangerous self-modification: just as a surgeon should not operate on themselves, the agent must not install or reinstall the VS Code extension it is running inside, since doing so would terminate its own process. The `SORCAR.md` override mechanism allows per-repository instructions to further customize the agent's behavior, forming a hierarchy: general system prompt → Sorcar-specific instructions → repository-specific `SORCAR.md`.

5 VS Code Extension: Unique Features

We ship our system as a VS Code extension. While the underlying agent architecture (Sections 3 and 4) already differs from existing AI coding assistants, the extension's user-facing design introduces

several features that, to our knowledge, are not present in any competing IDE or assistant—including GitHub Copilot [GitHub, 2021], Cursor [Cursor, 2024], Windsurf [Codeium, 2024], Devin [Cognition Labs, 2024], and Aider [Gauthier, 2023]. We enumerate these features below.

5.1 Git-Worktree Isolation with In-Editor Merge Review

Every mainstream AI coding assistant applies changes directly to the user's working tree. Copilot and Cursor insert completions or edits in place; Windsurf modifies files in the current branch; Aider auto-commits each change to the current branch. If the AI makes a bad edit, the developer must manually revert or cherry-pick.

Our extension never touches the user's working tree. Every task runs in a disposable git worktree on its own branch (Section 3.5). When the task completes, we present a per-hunk, per-file, and whole-task merge UI directly inside VS Code: the developer can accept or reject individual changes, navigate between hunks with dedicated commands, or accept/reject all changes in a file or across the entire task. This gives the developer the same granularity of control as a pull-request review, but embedded in the editor and operating on changes the AI just produced seconds ago.

No other assistant combines worktree-level isolation with an inline merge review flow. The closest analog is Aider's per-change auto-commit, but that operates on the current branch and provides no accept/reject UI—the developer must use standard git tooling to inspect and revert individual commits.

5.2 Automatic Multi-Session Continuation

Context windows are finite, and complex tasks—large refactorings, multi-file bug hunts, benchmark evaluations—routinely exceed them. Existing assistants handle this by either truncating the conversation (Copilot, Cursor) or asking the user to manually start a new session and re-describe the task (Windsurf, Aider). Devin operates in a sandboxed environment with a single long context, but when the context is exhausted, the task ends.

Our extension continues automatically. The Relentless Agent (Section 3.2) detects when the context window or step limit is approaching, serializes progress into a structured summary, and spawns a new sub-session that picks up where the previous one left off. If a sub-session crashes, a separate summarizer agent extracts useful context from the trajectory and feeds it to the next attempt. From the user's perspective, the task simply runs until it succeeds, regardless of how many sub-sessions are needed.

5.3 Cross-Session Self-Improvement

Every AI assistant starts each session from scratch: the user must re-explain project conventions, preferred patterns, and past decisions. Some tools support a static configuration file (e.g., `.cursorrules` for Cursor, `AGENTS.md` or `CLAUDE.md` for various agents), but these files must be written and maintained by the developer.

Our extension maintains a `USER_PREFS.md` file that the agent reads at the start of each task and *updates* at the end. When the agent discovers a new user preference or project invariant during task execution—a naming convention, an architectural constraint, a disliked pattern—it writes it to the file. Crucially, the agent also resolves conflicts: if a new preference contradicts an existing one, the old entry is removed. Over time, the file accumulates a curated set of project-specific knowledge that makes the agent progressively more effective without any manual configuration from the developer.

5.4 Real-Time Budget Accountability

AI coding assistants typically operate on a subscription model (Copilot, Cursor) or a per-seat pricing model (Devin, Windsurf) that obscures the per-task cost. The developer has no visibility into how many tokens a task consumed or how much it cost.

Our extension displays real-time cost tracking in the sidebar: input tokens, output tokens, cache hits, dollar cost, and elapsed time are updated at every agent step. We enforce both per-task and global budget ceilings. If a task exceeds its budget, the agent raises a hard error rather than silently

accumulating charges. This transparency allows developers to make informed decisions about which tasks to delegate to the AI and how to structure prompts for cost efficiency.

5.5 Multi-Tab Concurrent Tasks

Most AI coding assistants support only a single active task at a time. Copilot Chat, Cursor's agent mode, and Windsurf process one request before accepting the next. Devin can run tasks in the background but provides a separate web interface rather than IDE-native concurrency.

Our extension supports multiple concurrent tasks via a tabbed interface in the sidebar. Each tab runs an independent agent instance with its own worktree, context window, and budget tracking. The user can start a refactoring task in one tab, switch to another tab to ask an unrelated question, and return to find the refactoring still running. Per-tab state—DOM elements, pending user questions, merge UI—is preserved across tab switches by detaching and re-attaching per-tab DOM nodes rather than serializing and restoring values.

5.6 Integrated Browser Automation

While some assistants can search the web (Devin, Perplexity-based tools), none provide full programmatic browser control from within the IDE. Our extension includes a browser automation tool that allows the agent to navigate to URLs, read accessibility trees, click elements, type text, press keys, scroll, and take screenshots—all controlled programmatically from within a VS Code task. We render a live browser preview inside the extension's sidebar, allowing the developer to watch the agent interact with web applications in real time.

This capability enables use cases that no other IDE assistant supports: verifying a deployed web application after a code change, filling out web forms as part of a testing workflow, scraping documentation to inform a code generation task, or interacting with web-based developer tools (CI dashboards, issue trackers) without leaving the editor.

5.7 Crash Recovery from Git State

If VS Code crashes or the user restarts the editor mid-task, most AI assistants lose all context: the conversation is gone, and any partially applied changes are left in an indeterminate state in the working tree.

Our extension stores all worktree state in git itself—branch names, configuration entries, commit history—rather than in ephemeral sidecar files. On process restart, the agent queries git for any pending branches matching its chat ID prefix and reconstructs all instance attributes from git config. The user can resume reviewing and merging agent changes after a crash without any data loss.

5.8 Copilot Commit-Message Interception

VS Code's Source Control view exposes a sparkle button that invokes `github.copilot.git.generateCommitMessage` to generate a commit message using GitHub Copilot. Our extension registers its own handler for this command ID, intercepting the button so that our agent generates the commit message instead. This is a minor but practical feature: it allows users who do not have a Copilot subscription to still use the sparkle button, powered by whatever LLM backend the extension is configured to use.

6 Preliminary Evaluation on Terminal Bench 2.0

We evaluate our system on Terminal Bench 2.0,[1] a benchmark of 89 diverse terminal-based programming tasks that range from building legacy compilers and configuring servers to solving cryptanalysis challenges and training machine-learning models. Each task runs inside an isolated Docker container; a separate verifier judges the result automatically. We use the Harbor[2] framework to orchestrate execution, and Claude Opus 4.6 as the underlying LLM.

[1] `https://www.tbench.ai/`

[2] `https://github.com/harbor-framework/harbor`

6.1 Setup

We run 4 independent trials per task (totaling 330 scored trials after excluding 3 that produced no verifier output). The agent is `SorcarHarborAgent`, a thin Harbor adapter that installs and invokes the sorcar CLI inside each container. We hard-skip 9 tasks that we verified to be infeasible for Opus 4.6 across 6+ prior attempts (e.g. CompCert compilation, Windows 3.11 GUI installation, video OCR). Skipped tasks still count as failures.

6.2 Aggregate Results

Table 1 summarizes the aggregate statistics.

Table 1: Terminal Bench 2.0 aggregate results (89 tasks, 4 trials each, Claude Opus 4.6).

Metric	**Value**
Total tasks	89
Total scored trials	330
Overall pass rate	62.1% (205/330)
pass@any (at least 1/4 passes)	75.3% (67/89)
pass@all (all 4 pass)	42.7% (38/89)
Always-fail tasks	22
Always-pass tasks	38
Mixed-result tasks	29
Median cost per trial	$0.40
Mean cost per trial	$0.91
Median duration per trial	202 s
Mean duration per trial	467 s

The 62.1% overall pass rate compares favorably to other agents using the same underlying model: at the time of writing, Claude Code (also Opus 4.6) scores approximately 58% on the Terminal Bench 2.0 leaderboard, and the Cursor agent—which uses a custom fine-tuned model—achieves roughly 62%. Our result suggests that the layered architecture and the structured system prompt described in Sections 3–4 contribute meaningfully beyond what the base model provides.

6.3 Task-Level Breakdown

Consistently solved tasks (38 of 89). These include cryptanalysis (FEAL linear and differential), git operations (multi-branch merge, leak recovery), server configuration (gRPC key-value store, PyPI server, NGINX logging), data processing (resharding, large-scale text editing), formal verification (Coq `plus_comm`), and ML inference (HuggingFace model serving, LLM batching scheduler). The breadth of this set—spanning systems, security, data engineering, and formal methods—demonstrates that the agent's generality is not limited to a single domain.

Consistently failed tasks (22 of 89). The failures cluster into three categories: (1) tasks requiring graphical or multimedia capabilities unavailable in the container (video processing, Windows 3.11 GUI, MTEB leaderboard scraping), (2) tasks demanding very long or resource-intensive builds that exceed the container's time or memory limits (CompCert, Doom for MIPS, Caffe CIFAR-10, training fastText on Yelp data), and (3) tasks with niche domain-specific requirements that the model struggles to satisfy (DNA assembly, OCaml GC patching, polyglot C/Rust binaries).

Mixed-result tasks (29 of 89). Tasks such as `chess-best-move` (3/4), `write-compressor` (3/4), and `qemu-startup` (3/4) succeed in most trials but occasionally fail due to non-determinism in the model's reasoning or timing-sensitive environment interactions. Conversely, `configure-git-webserver` (1/4) and `protein-assembly` (1/4) succeed rarely, suggesting they are at the boundary of the model's capability.

6.4 Discussion

These results are preliminary: the benchmark is still running and we have not yet completed the 5 trials per task required for official leaderboard submission. Nevertheless, several observations stand out:

- **System prompt matters.** An earlier version of our agent using only a basic prompt achieved approximately 61% on Terminal Bench. The improved prompt presented in this paper raises that to 67% in our latest configuration run (66.2% with 2 trials per task), confirming that the prompt engineering described in Section 4 has measurable impact.
- **Variance is high.** The gap between pass@any (75.3%) and pass@all (42.7%) shows that many tasks are solvable in principle but not reliably. This variance is inherent to the underlying LLM and is not specific to our agent.
- **Cost is modest.** At a median of $0.40 per trial and $0.91 mean, the agent is practical for routine use. The total cost for the 330-trial run was approximately $240.
- **Leaderboard context.** Anthropic has noted on social media that approximately 81% represents the feasibility ceiling for Terminal Bench 2.0. Recent analysis has also shown that several top-scoring agents on the leaderboard employ benchmark-specific optimizations that inflate their scores.

7 Related Work

Code generation agents. SWE-Agent [Yang et al., 2024] and OpenHands [Wang et al., 2024] provide LLM-based agents for resolving software engineering tasks such as GitHub issues. Both use a single-session execution model without automatic continuation. Agentless [Xia et al., 2024] takes the opposite approach, showing that a simple two-phase localize-then-repair pipeline without autonomous agent loops can achieve competitive results on SWE-bench [Jimenez et al., 2024]. Devin [Cognition Labs, 2024], an industrial product marketed as "the first AI software engineer," operates in a sandboxed environment with shell, browser, and editor access. Aider [Gauthier, 2023] provides a terminal-based pair programming interface with tight git integration, automatically committing each change. Our Relentless Agent layer addresses the single-session limitation shared by most of these systems, while our worktree isolation provides stronger safety guarantees than per-change commits.

ReAct and tool use. The ReAct framework [Yao et al., 2023] interleaves reasoning and action. Toolformer [Schick et al., 2023] teaches models to use tools via self-supervised learning. We use native function calling provided by modern LLM APIs rather than in-context tool descriptions, which reduces prompt overhead and improves reliability.

Reasoning and planning. Chain-of-thought prompting [Wei et al., 2022] demonstrated that eliciting step-by-step reasoning dramatically improves LLM performance on complex tasks. Tree of Thoughts [Yao et al., 2024] generalizes this to deliberate search over multiple reasoning paths. Reflexion [Shinn et al., 2023] introduces verbal reinforcement learning, where an agent reflects on failed attempts and produces self-critiques that improve subsequent trials. Our continuation protocol is conceptually related to Reflexion: failed sub-sessions produce summaries that inform subsequent attempts. However, we operate at the level of full software engineering tasks rather than isolated reasoning problems, and its budget and step limits provide principled termination criteria.

Multi-agent software development. ChatDev [Qian et al., 2024] models the software development process as a conversation between role-playing agents (CEO, CTO, programmer, tester) organized in a waterfall-like pipeline. MetaGPT [Hong et al., 2024] takes a meta-programming approach, encoding standard operating procedures as structured outputs that coordinate specialized agents. AutoGen [Wu et al., 2023] provides a general-purpose framework for multi-agent conversation, enabling flexible topologies beyond fixed pipelines. These systems focus on generating entire applications from scratch. We take a different approach: rather than simulating an organization of specialists, we use a single agent with broad tool access and optional parallel sub-agents for embarrassingly parallel sub-tasks, prioritizing practical utility on real-world codebases over role-playing fidelity.

Multi-turn autonomous agents. AutoGPT [Significant Gravitas, 2023] and BabyAGI [Nakajima, 2023] implement multi-step autonomous agents. These systems typically lack budget controls and safe code isolation. Our layered architecture addresses each of these concerns in a dedicated layer.

Code-specialized language models. Beyond the general-purpose LLMs that our system can use as backends, a rich line of work has produced models specifically trained for code. Code Llama [Rozière et al., 2023] fine-tunes Llama 2 for code generation and infilling. StarCoder [Li et al., 2023] trains on permissively licensed code from GitHub with a fill-in-the-middle objective. DeepSeek-Coder [Guo et al., 2024] trains on a 2-trillion-token corpus of code and natural language with a repository-level context window. Our system is model-agnostic and can leverage any of these models through its pluggable LLM backend, benefiting from advances in code-specialized pre-training without architectural changes.

Software engineering benchmarks. SWE-bench [Jimenez et al., 2024] evaluates agents on real-world GitHub issues drawn from popular Python repositories, requiring the agent to localize and fix bugs given only the issue description. It has become the de facto standard for measuring agent capabilities in software engineering. HumanEval [Chen et al., 2021] and MBPP [Austin et al., 2021] evaluate function-level code generation from docstrings. These benchmarks focus on isolated coding problems; We target the broader workflow of multi-file, multi-step software engineering tasks that require tool use, testing, and version control.

Agent frameworks and orchestration. LangChain [LangChain, 2022] provides modular abstractions for building LLM applications, including agent loops, tool integration, and memory. It focuses on composability and breadth of integrations rather than on the specific concerns of software development. Our architecture is purpose-built for software engineering, with each layer addressing a specific practical concern (budget tracking, continuation, code safety) rather than providing general-purpose primitives.

LLM agent surveys. Several comprehensive surveys have mapped the rapidly growing landscape of LLM-based agents. Xi et al. [2023] survey the design space of LLM agents along three dimensions — brain (reasoning), perception (input modalities), and action (tool use) — and catalog applications across social science, natural science, and engineering. Wang et al. [2024b] propose a systematic framework for autonomous agents built on LLMs, covering profile, memory, planning, and action modules. Our system can be understood through the lens of these frameworks: the KISS Agent implements the action loop, the Relentless Agent addresses the memory and planning concerns through continuation summaries, and the system prompt encodes the profile.

IDE integration. GitHub Copilot [GitHub, 2021], Cursor [Cursor, 2024], and Windsurf [Codeium, 2024] provide AI assistance within editors. These tools typically operate at the level of code completion or single-turn chat. We operate at the level of autonomous multi-step task execution with full tool access and git-level isolation.

Recent advances in agentic software engineering. The field has matured rapidly since late 2025. Hassan et al. [2025] articulate the foundational pillars of *Agentic Software Engineering* (SE 3.0), defining a research roadmap that emphasizes trust, controllability, and goal-oriented task decomposition. Li et al. [2025] survey the landscape of autonomous coding agents and characterize the transition from assistive code completion to full-fledged AI teammates that initiate, plan, and execute development tasks. We instantiate several of the principles advocated in these roadmaps, including layered controllability (via budget tracking and step limits) and safe isolation (via worktrees).

Wang et al. [2025] provide a comprehensive survey of AI agentic programming techniques, cataloging how LLM-based agents decompose goals, interact with compilers and version control systems, and self-correct through feedback loops. Chatlatanagulchai et al. [2025] study *agent context files* — persistent, project-level instruction files that guide agentic coding tools — and find that high-quality context files significantly improve agent performance. Our layered system prompt and `SORCAR.md` override mechanism are instances of this pattern. Mohsenimofidi et al. [2025] further investigate context engineering for AI agents in open-source software, highlighting how curated context improves agent efficacy on repository-level tasks.

Evolving benchmarks for coding agents. SWE-bench Pro [Deng et al., 2025] introduces a substantially more challenging benchmark with 1,865 long-horizon problems drawn from 41 repositories, including commercial codebases, explicitly targeting enterprise-level complexity beyond the original SWE-bench. These long-horizon tasks — often requiring multi-file patches and hours of professional developer effort — directly motivate our continuation mechanism. Prathifkumar et al. [2025] raise the important question of benchmark contamination, showing that overlap between SWE-Bench-Verified problems and LLM training data may inflate scores, suggesting that high benchmark numbers may

partly reflect memorization rather than genuine problem-solving ability. Horikawa et al. [2025] provide an empirical study of how AI coding agents handle refactoring tasks, finding that while agents can plan and execute complex refactorings, they still struggle with cross-file dependency analysis — a challenge that our sub-agent parallelism partially addresses.

Self-improvement and test-time scaling. Robeyns et al. [2025] demonstrate a self-improving coding agent that iteratively refines its own scaffolding code, achieving dramatic benchmark improvements through self-generated optimizations. Our self-improvement loop (via `USER_PREFS.md`) captures a lighter-weight form of this idea: rather than rewriting its own code, the agent accumulates learned user preferences and project invariants across sessions. Gao et al. [2025] introduce the Trae Agent, which applies test-time compute scaling to software engineering, dynamically allocating more inference budget to harder problems. Our budget-tracking mechanism provides a complementary perspective: rather than scaling compute adaptively, we enforce hard budget ceilings while the Relentless Agent ensures maximum progress within those bounds.

8 Conclusion

We have shown that a layered, single-concern architecture can address the practical challenges of deploying LLM agents for real-world software development. The KISS Agent provides a robust execution core with budget tracking. The Relentless Agent eliminates the context-window ceiling through automatic continuation. The Sorcar Agent equips the system with the tools a developer actually uses. The Chat Sorcar Agent enables multi-turn conversations that persist across sessions. The Worktree Sorcar Agent ensures that every task is safely isolated in git, giving the developer full control over what gets merged.

We complement the architecture with a system prompt that encodes engineering discipline directly into the agent's behavior: read before writing, test before fixing, plan before executing, verify before finishing. These are not novel insights — they are the practices of careful software engineering, translated into instructions that an LLM can follow.

We make the source code publicly available at `https://github.com/ksenxx/kiss_ai`.

References

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Yang, J., Jimenez, C. E., Wettig, A., Liber, K., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., and Ji, H. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Significant Gravitas. AutoGPT. `https://github.com/Significant-Gravitas/AutoGPT`, 2023.

Nakajima, Y. BabyAGI. `https://github.com/yoheinakajima/babyagi`, 2023.

GitHub. GitHub Copilot: Your AI pair programmer. `https://github.com/features/copilot`, 2021.

Cursor. Cursor: The AI-first code editor. `https://cursor.sh`, 2024.

Codeium. Windsurf: The AI-powered IDE. `https://windsurf.com`, 2024.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

Cognition Labs. Devin: The first AI software engineer. `https://devin.ai`, 2024.

Gauthier, P. Aider: AI pair programming in your terminal. `https://github.com/paul-gauthier/aider`, 2023.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., and Sun, M. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., and Schmidhuber, J. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Li, R., Allal, L. B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al. StarCoder: May the source be with you! *Transactions on Machine Learning Research (TMLR)*, 2023.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y. K., Luo, F., Xiong, Y., and Liang, W. DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

LangChain. LangChain: Build context-aware reasoning applications. `https://github.com/langchain-ai/langchain`, 2022.

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.

Hassan, A. E., Li, H., Lin, D., and Adams, B. Agentic software engineering: Foundational pillars and a research roadmap. *arXiv preprint arXiv:2509.06216*, 2025.

Li, H., Zhang, H., and Hassan, A. E. The rise of AI teammates in software engineering (SE 3.0): How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003*, 2025.

Deng, X., Da, J., Pan, E., He, Y. Y., Ide, C., Garg, K., Lauffer, N., Park, A., Pasari, N., Rane, C., et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

Prathifkumar, T., Mathews, N. S., and Nagappan, M. Does SWE-Bench-Verified test agent ability or model memory? *arXiv preprint arXiv:2512.10218*, 2025.

Horikawa, K., Li, H., Kashiwa, Y., and Adams, B. Agentic refactoring: An empirical study of AI coding agents. *arXiv preprint arXiv:2511.04824*, 2025.

Wang, H., Gong, J., Zhang, H., Xu, J., and Wang, J. AI agentic programming: A survey of techniques, challenges, and opportunities. *arXiv preprint arXiv:2508.11126*, 2025.

Chatlatanagulchai, W., Li, H., Kashiwa, Y., and Reid, B. Agent READMEs: An empirical study of context files for agentic coding. *arXiv preprint arXiv:2511.12884*, 2025.

Mohsenimofidi, S., Galster, M., and Treude, C. Context engineering for AI agents in open-source software. *arXiv preprint arXiv:2510.21413*, 2025.

Robeyns, M., Szummer, M., and Aitchison, L. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.

Gao, P., Tian, Z., Meng, X., and Trae Research Team. Trae Agent: An LLM-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025.

GetShitDone. GetShitDone: Prompt engineering for coding agents. `https://github.com/alexnodejs/get-shit-done`, 2025.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.